

Smart Lender: Applicant Credibility Prediction for Loan Approval

Project Description:

Smart Lender leverages Machine Learning to automate and enhance the loan approval process by predicting the creditworthiness of loan applicants. The platform analyzes applicant information, including gender, marital status, education, employment status, applicant income, co-applicant income, loan amount, loan amount term, credit history, and property area, to determine whether a loan application is likely to be approved or rejected. The system incorporates multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, to build and evaluate predictive models. After comparing their performance using accuracy, confusion matrix, classification report, and cross-validation, the best-performing model is selected and saved for deployment.

Built using Python and the Flask web framework, the Smart Lender application provides a user-friendly web interface where applicants or banking professionals can enter loan details and receive instant loan eligibility predictions. The system integrates the trained machine learning model with the Flask backend using Pickle for model loading and is designed for deployment on IBM Cloud. By enabling fast, consistent, and data-driven loan approval decisions, Smart Lender helps financial institutions reduce manual effort, improve operational efficiency, and support reliable credit evaluation.

Scenarios

Scenario 1: Fast Loan Approval for Eligible Applicant

A bank credit officer enters the details of an applicant with a stable income, good credit history, and suitable loan amount through the Smart Lender web application. The system processes the information using the trained XGBoost model and instantly predicts that the loan is **Approved**, enabling faster and more efficient loan processing.

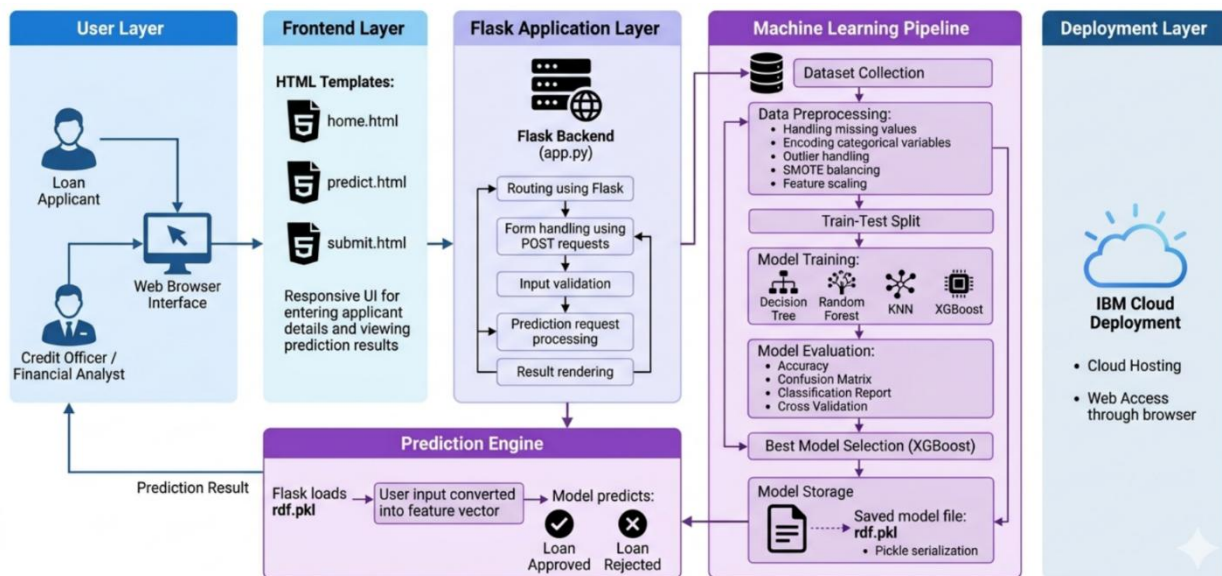
Scenario 2: High-Risk Applicant Detection

An applicant with irregular income, poor credit history, and a high loan amount submits a loan application through the system. Smart Lender analyzes the applicant's information and predicts that the loan is **Rejected**, allowing the financial institution to identify high-risk applications and reduce the possibility of loan defaults.

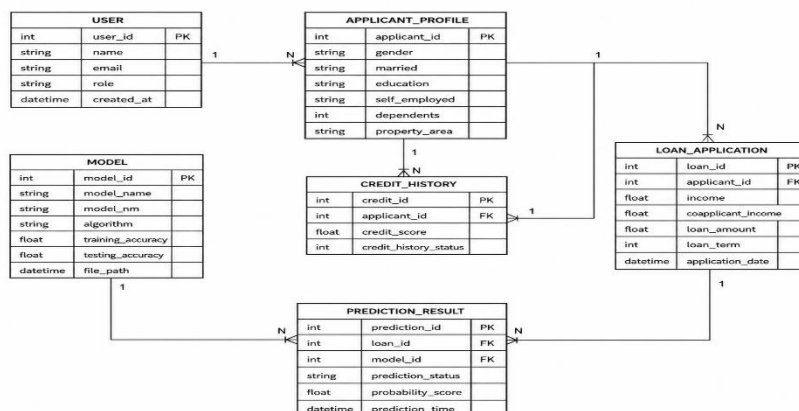
Scenario 3: Efficient Loan Evaluation for Banking Professionals

A financial analyst receives multiple loan applications during a busy period. Instead of manually reviewing every application, the analyst uses the Smart Lender system to evaluate each applicant's eligibility. The application provides instant loan approval predictions, helping reduce processing time, improve consistency, and support data-driven lending decisions.

Technical Architecture:



Description: Smart Lender utilizes a modular three-tier architecture to provide fast and accurate loan eligibility predictions using machine learning. The Presentation Layer offers a user-friendly web interface built with HTML and CSS, allowing applicants and banking professionals to enter loan details easily. The Application Layer, developed using the Flask framework, manages user requests, validates input data, and communicates with the trained machine learning model. The Machine Learning Layer loads the saved XGBoost model (selected after evaluating Decision Tree, Random Forest, and KNN models) using Pickle to generate real-time loan approval predictions. The application is designed for deployment on IBM Cloud, enabling secure, scalable, and efficient access to the Smart Lender system.



Pre-requisites:

- **Python Programming Knowledge:** Understanding of Python programming concepts, syntax, functions, and object-oriented programming. (*Python Documentation*)
- **Machine Learning Fundamentals:** Knowledge of supervised learning, classification algorithms, model evaluation, and feature engineering using Scikit-learn.
- **Flask Framework Knowledge:** Familiarity with building web applications using the Flask framework. (*Flask Documentation*)
- **HTML and CSS Skills:** Basic knowledge of HTML and CSS to develop the frontend user interface. (*W3Schools HTML/CSS Tutorials*)
- **Data Analysis Libraries:** Experience with **NumPy**, **Pandas**, **Matplotlib**, and **Seaborn** for data preprocessing, analysis, and visualization.
- **Model Serialization:** Understanding of **Pickle** for saving and loading trained machine learning models.
- **Version Control with Git and GitHub:** Basic knowledge of Git commands and GitHub for source code management and collaboration.
- **IBM Cloud Basics:** Familiarity with deploying Flask applications on IBM Cloud (or a similar cloud platform).

Project Workflow

Milestone 1: Data Collection and Architecture Design

- **Activity 1.1:** Download the Loan Prediction dataset from the provided source and store it in the project directory.
- **Activity 1.2:** Study the dataset and identify the input features and target variable required for loan approval prediction.
- **Activity 1.3:** Design the application architecture showing the interaction between the frontend, Flask backend, machine learning model, and prediction output.
- **Activity 1.4:** Set up the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, and other required libraries.

Milestone 2: Data Analysis and Preprocessing

- **Activity 2.1:** Import the dataset using Pandas and perform Exploratory Data Analysis (EDA), including univariate, bivariate, and multivariate analysis.
- **Activity 2.2:** Handle missing values, encode categorical variables, balance the dataset using SMOTE, perform feature scaling, and split the dataset into training and testing sets.

Milestone 3: Machine Learning Model Development

- **Activity 3.1:** Train Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost classification models using the prepared dataset.
- **Activity 3.2:** Evaluate all models using accuracy score, confusion matrix, classification report, and cross-validation.
- **Activity 3.3:** Save the best-performing machine learning model as a .pkl file for deployment.

Milestone 4: Flask Application Development

- **Activity 4.1:** Develop the Flask backend to load the trained machine learning model and process user input.
- **Activity 4.2:** Design the frontend using HTML and CSS, including **home.html**, **predict.html**, and **submit.html** pages.
- **Activity 4.3:** Integrate the frontend with the Flask backend to generate real-time loan eligibility predictions.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the complete Smart Lender application to verify accurate predictions and smooth functionality.
- **Activity 5.2:** Prepare the application for deployment and deploy the Flask application on **IBM Cloud**.

Milestone 6: Project Conclusion

- **Activity 6.1:** Validate the final application, prepare the project documentation, and demonstrate the Smart Lender system for evaluation.

Milestone 1: Data Collection and Architecture Design

In this milestone, the project begins with collecting the loan prediction dataset and designing the overall system architecture. The dataset is analyzed to identify the input features and target variable required for loan eligibility prediction. A modular three-tier architecture is designed to establish communication between the frontend, Flask backend, and the machine learning model. The development environment is then configured by installing the required Python libraries and frameworks to support data preprocessing, model training, and web application development.

Activity 1.1: Download and Prepare the Dataset

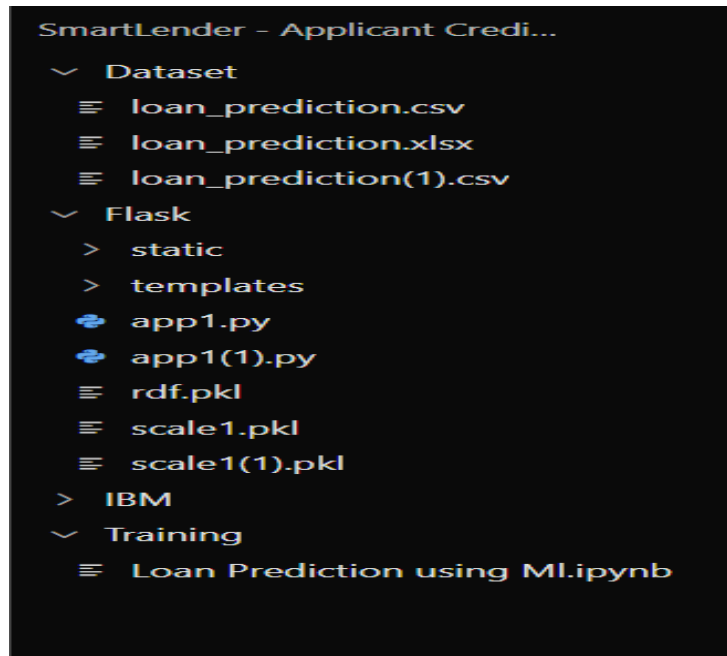
Before developing the machine learning model, the loan prediction dataset must be downloaded and stored in the project directory. This dataset contains applicant information such as gender, marital status, education, employment status, applicant income, co-applicant income, loan amount, loan amount term, credit history, property area, and loan approval status.

Steps:

- **Step 1 - Download the Dataset:** Download the Loan Prediction dataset from the provided project link or the Google Drive repository shared in SkillWallet.

loan_prediction														
File Edit View Insert Format Data Tools Help														
Q Menus 100% Arial 10 B I														
A1	Loan_ID													
	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status	
2	LP001002	Male	No		0 Graduate	No	5849	0	360	1 Urban	Y			
3	LP001003	Male	Yes		1 Graduate	No	4583	1508	128	360	1 Rural	N		
4	LP001005	Male	Yes		0 Graduate	Yes	3000	0	96	360	1 Urban	Y		
5	LP001006	Male	Yes		0 Not Graduate	No	2563	2358	120	360	1 Urban	Y		
6	LP001008	Male	No		0 Graduate	No	6000	0	141	360	1 Urban	Y		
7	LP001011	Male	Yes		2 Graduate	Yes	5417	4196	267	360	1 Urban	Y		
8	LP001013	Male	Yes		0 Not Graduate	No	2333	1516	95	360	1 Urban	Y		
9	LP001014	Male	Yes		Graduate	No	3036	2504	158	360	0 Semiurban	N		
10	LP001019	Male	Yes	3+	2 Graduate	No	4006	1526	168	360	1 Urban	Y		
11	LP001020	Male	Yes		1 Graduate	No	12841	10968	349	360	1 Semiurban	N		
12	LP001024	Male	Yes		2 Graduate	No	3200	700	70	360	1 Urban	Y		
13	LP001027	Male	Yes		2 Graduate	No	2500	1840	109	360	1 Urban	Y		
14	LP001028	Male	Yes		2 Graduate	No	3073	8106	200	360	1 Urban	Y		
15	LP001029	Male	No		0 Graduate	No	1853	2840	114	360	1 Rural	N		
16	LP001030	Male	Yes		2 Graduate	No	1299	1086	17	120	1 Urban	Y		
17	LP001032	Male	No		0 Graduate	No	4950	0	125	360	1 Urban	Y		
18	LP001034	Male	No		1 Not Graduate	No	3595	0	100	240	Urban	Y		
19	LP001036	Female	No		0 Graduate	No	3510	0	76	360	0 Urban	N		
20	LP001038	Male	Yes		0 Not Graduate	No	4887	0	133	360	1 Rural	N		
21	LP001041	Male	Yes		0 Graduate	No	2600	3500	115		1 Urban	Y		
22	LP001043	Male	Yes		0 Not Graduate	No	7660	0	104	360	0 Urban	N		
23	LP001046	Male	Yes		1 Graduate	No	5955	5625	315	360	1 Urban	Y		
24	LP001047	Male	Yes		0 Not Graduate	No	2600	1911	115	360	0 Semiurban	N		
25	LP001050	Yes			2 Not Graduate	No	3365	1917	112	360	0 Rural	N		
26	LP001052	Male	Yes		1 Graduate	No	3717	2925	151	360	Semiurban	N		
27	LP001066	Male	Yes		0 Graduate	Yes	9560	0	191	360	1 Semiurban	Y		

Step 2 – Create the Project Directory: Create a project folder and store the dataset (loan_prediction.csv) inside the project directory.



Step 3 – Verify the Dataset: Open the dataset using Pandas and verify that all required features and the target variable are available.

```
[ ]
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV
import imblearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
```

Step 4 – Prepare the Development Environment: Install the required Python libraries, including NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Imbalanced-learn (SMOTE), Flask, XGBoost, and Pickle.

```
[ ] ▶ #importing the dataset which is in csv file
data = pd.read_csv('/content/loan_prediction.csv')
data
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	
...	...	...	...	...	...	...	...	...	...	...	...
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	360.0	
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	180.0	
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	360.0	
612	LP002984	Male	Yes	0	Graduate	No	7583	0.0	127.0	360.0	

Step 5 – Organize the Project Structure: Create separate folders for datasets, machine learning models, templates, static files, and the Flask application to maintain a clean and scalable project structure.

Activity 1.2: Research and Select the Appropriate Machine Learning Models

The following activities were performed to identify the most suitable machine learning algorithm for predicting loan eligibility:

- **Understand the Project Requirements:** Analyze the project objective and identify the input features such as applicant income, co-applicant income, loan amount, loan term, education, employment status, credit history, property area, and other applicant details required for loan approval prediction.
- **Explore Machine Learning Algorithms:** Study different supervised classification algorithms, including **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **XGBoost**, to understand their working principles, advantages, limitations, and suitability for classification problems.
- **Evaluate Model Performance:** Train each model using the prepared dataset and compare their performance using evaluation metrics such as **training accuracy**, **testing accuracy**, **confusion matrix**, **classification report**, and **cross-validation score**.
- **Select the Best Model:** Based on the evaluation results, select the **XGBoost** classifier as the final model because it achieved the highest prediction accuracy and demonstrated better generalization compared to the other machine learning models. The trained model is then saved using **Pickle** for integration with the Flask web application.

Activity 1.3: Define the Architecture of the Application

The Smart Lender application follows a modular three-tier architecture consisting of the **Presentation Layer**, **Application Layer**, and **Machine Learning Layer**. This architecture ensures efficient communication between the user interface, backend application, and predictive model while maintaining scalability and ease of maintenance.

- **Draft an Architectural Diagram:** Create a high-level architecture diagram illustrating the interaction between the HTML/CSS frontend, Flask backend, trained machine learning model, and prediction output. The diagram should clearly represent the flow of data from user input to prediction result.
- **Detail Frontend Functionality:** Describe how users interact with the application by entering applicant information such as gender, marital status, education, employment status, applicant income, co-applicant income, loan amount, loan amount term, credit history, and property area through the web interface.
- **Outline Backend Responsibilities:** Explain how the Flask backend receives user input through HTTP POST requests, validates and preprocesses the data, loads the trained machine learning model, performs prediction, and returns the loan approval result to the user interface.
- **Describe Machine Learning Integration:** Define how the Flask application loads the saved **XGBoost** model using **Pickle**, processes the input features in the required format, generates the loan eligibility prediction, and displays the prediction (Approved/Rejected) on the results page.

Activity 1.4: Set Up the Development Environment

Before developing the Smart Lender application, the required software, libraries, and project structure must be configured to support machine learning model development and Flask-based web application deployment.

- Install Python and Pip

Ensure **Python 3.8 or above** is installed on your system along with **pip**, which is used to install the required Python packages.

Create a Virtual Environment

Create and activate a virtual environment to isolate the project dependencies.

```
D:\Projects> python -m venv venv
```

```
D:\Projects> venv\Scripts\activate
```

After activation, install all required packages.

```
(venv) D:\Projects> pip install -r requirements.txt
```

Install Flask

Install the Flask framework to build the web application.

• Install Machine Learning Libraries

Install the required libraries for data analysis, visualization, preprocessing, model training, and deployment.

**(venv) D:\Projects> pip install pandas numpy matplotlib seaborn
scikit-learn xgboost imbalanced-learn**

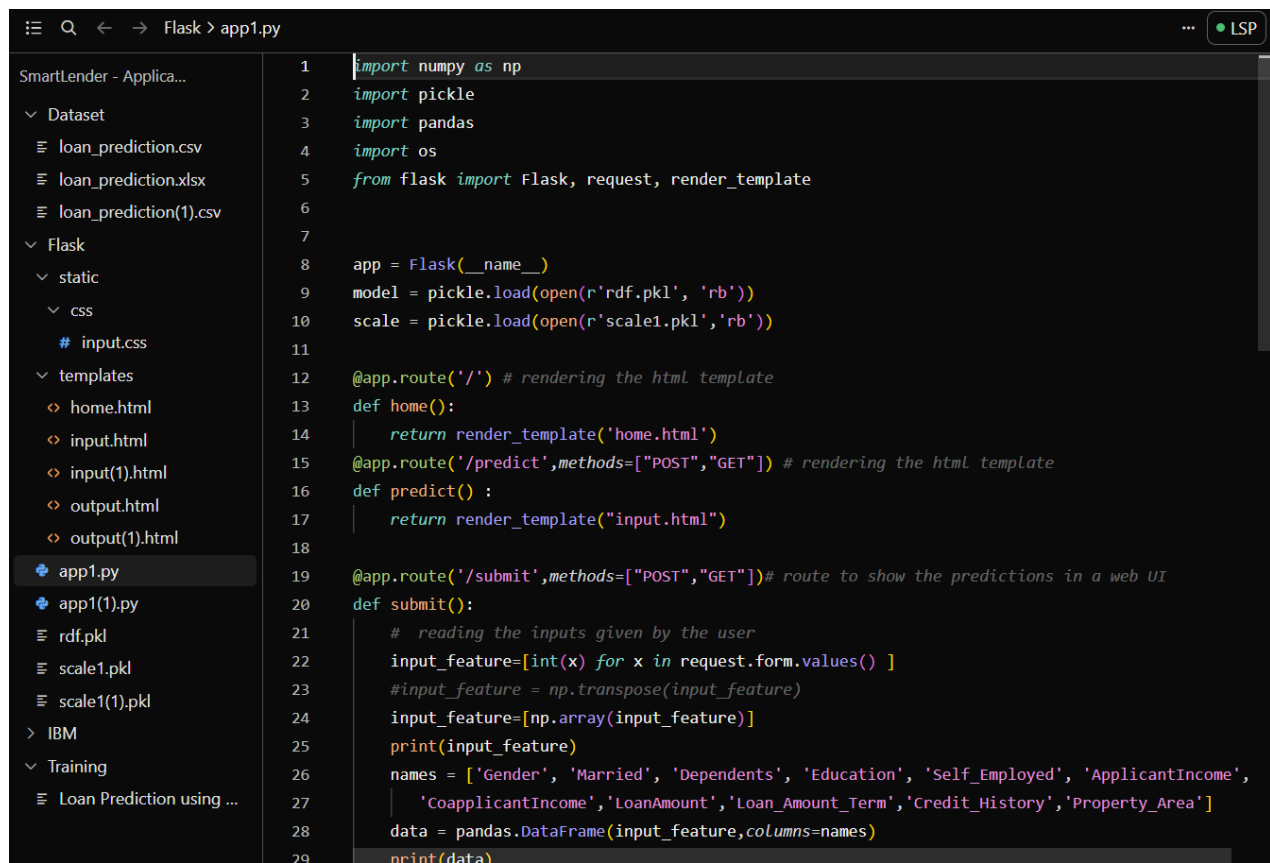
• Install Additional Packages

Install Pickle (built into Python) for saving/loading trained models and other required dependencies if necessary.

(venv) D:\Projects> pip install joblib

• Set Up the Project Structure

Create the project directory with separate folders for datasets, machine learning models, HTML templates, static files, and the Flask application.



```

1  import numpy as np
2  import pickle
3  import pandas
4  import os
5  from flask import Flask, request, render_template
6
7
8  app = Flask(__name__)
9  model = pickle.load(open(r'rdf.pkl', 'rb'))
10 scale = pickle.load(open(r'scale1.pkl', 'rb'))
11
12 @app.route('/') # rendering the html template
13 def home():
14     return render_template('home.html')
15 @app.route('/predict', methods=["POST", "GET"]) # rendering the html template
16 def predict():
17     return render_template("input.html")
18
19 @app.route('/submit', methods=["POST", "GET"]) # route to show the predictions in a web UI
20 def submit():
21     # reading the inputs given by the user
22     input_feature=[int(x) for x in request.form.values() ]
23     #input_feature = np.transpose(input_feature)
24     input_feature=np.array(input_feature)]
25     print(input_feature)
26     names = [ 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome',
27             'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area' ]
28     data = pandas.DataFrame(input_feature, columns=names)
29     print(data)
  
```


Milestone 2: Data Analysis and Preprocessing

Activity 2.1: Perform Data Analysis and Visualization

Analyze the loan prediction dataset to understand the characteristics of the data before building the machine learning models.

Import the Dataset

- Load the loan prediction dataset using the **Pandas** library.
- Display the first few records using `df.head()` to verify successful data loading.

Perform Univariate Analysis

- Analyze the distribution of individual features using visualization techniques.
- Use **count plots** for categorical variables such as Gender, Education, and Credit History.
- Use **distribution plots** or **histograms** for numerical variables such as Applicant Income and Loan Amount.

Perform Bivariate Analysis

- Analyze the relationship between two variables.
- Compare applicant features with the loan approval status using count plots, box plots, or bar charts.

Perform Multivariate Analysis

- Analyze relationships among multiple features simultaneously.
- Use correlation heatmaps and pair plots to identify important features influencing loan approval.

Activity 2.2: Perform Data Preprocessing

Prepare the dataset for machine learning by cleaning and transforming the data.

Handle Missing Values

- Identify missing values using `df.isnull().sum()`.
- Replace missing numerical values with the **mean**.
- Replace missing categorical values with the **mode**.

Encode Categorical Variables

- Convert categorical features such as Gender, Married, Education, Self_Employed, and Property_Area into numerical values using appropriate encoding techniques.

Balance the Dataset

- Apply **SMOTE (Synthetic Minority Over-sampling Technique)** to balance the target classes and improve model performance.

Feature Scaling

- Scale numerical features using **StandardScaler** to improve the performance of distance-based algorithms such as KNN.

Split the Dataset

- Divide the dataset into **training** and **testing** sets using `train_test_split()` for model training and evaluation.
-

Milestone 3: Machine Learning Model Development

Milestone 3 focuses on building, training, evaluating, and saving the machine learning models used for loan approval prediction. Multiple classification algorithms are trained and compared to identify the best-performing model. The selected model is then saved and prepared for integration with the Flask web application.

Activity 3.1: Develop the Machine Learning Models

Train the Decision Tree Model

- Import the **DecisionTreeClassifier** from Scikit-learn.

- Train the model using the training dataset. Predict loan approval status using the testing dataset.
- Evaluate the model using the confusion matrix, classification report, and accuracy score.

```
[ ]  #importing and building the Decision tree model
def decisionTree(X_train,X_test,y_train,y_test):
    model = DecisionTreeClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
decisionTree(X_train,X_test,y_train,y_test)
```

•

Train the Random Forest Model

- Import the **RandomForestClassifier**.
- Train the model using the same training dataset.
- Compare its prediction performance with the Decision Tree model.

```
[ ]  #importing and building the random forest model
def RandomForest(X_train,X_test,y_train,y_test):
    model = RandomForestClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
RandomForest(X_train,X_test,y_train,y_test)
```

Train the K-Nearest Neighbors (KNN) Model

- Import the **KNeighborsClassifier**.
- Train the model using the scaled dataset.
- Evaluate its performance using accuracy and classification metrics.

```
[ ] #importing and building the KNN model
def KNN(X_train,X_test,y_train,y_test):
    model = KNeighborsClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
KNN(X_train,X_test,y_train,y_test)
```

Train the XGBoost Model

- Import the **XGBClassifier**.
- Train the model using the prepared dataset.
- Compare its performance with the other machine learning algorithms.

```
[ ] #importing and building the Xg boost model
def XGB(X_train,X_test,y_train,y_test):
    model = GradientBoostingClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
XGB(X_train,X_test,y_train,y_test)
```

Evaluate Model Performance

- Compare all trained models using:
 - Accuracy Score
 - Confusion Matrix
 - Classification Report
 - Cross-validation Score
- Select the best-performing model based on testing accuracy and overall performance.

▼ Saving the Model

[]

```
#saviung the model by using pickle function
pickle.dump(model,open('rdf.pkl','wb'))
```

[]

```
Start coding or generate with AI.
```

Save the Best Model

- Save the selected model using the **Pickle** library.
- The saved model (.pkl file) will later be loaded by the Flask application for real-time loan eligibility prediction.

Milestone 4: Flask Application and Frontend Development

Milestone 4 focuses on developing a user-friendly web application for the Smart Lender system. This involves designing responsive HTML pages for loan eligibility prediction and integrating them with the Flask backend to provide real-time loan approval predictions using the trained machine learning model.

Activity 4.1: Designing and Developing the User Interface

Set Up the HTML Pages

- Develop three HTML pages:
 - **home.html** – Displays the project title, description, and a **Predict Loan** button.
 - **predict.html** – Provides a form where users can enter applicant details required for loan prediction.
 - **submit.html** – Displays the loan prediction result (Approved or Rejected).
- Use semantic HTML elements such as **header**, **section**, **form**, **label**, and **button** to organize the application and improve accessibility.
- Design a simple and professional interface suitable for banking and financial applications.

Design a Responsive Layout Using CSS

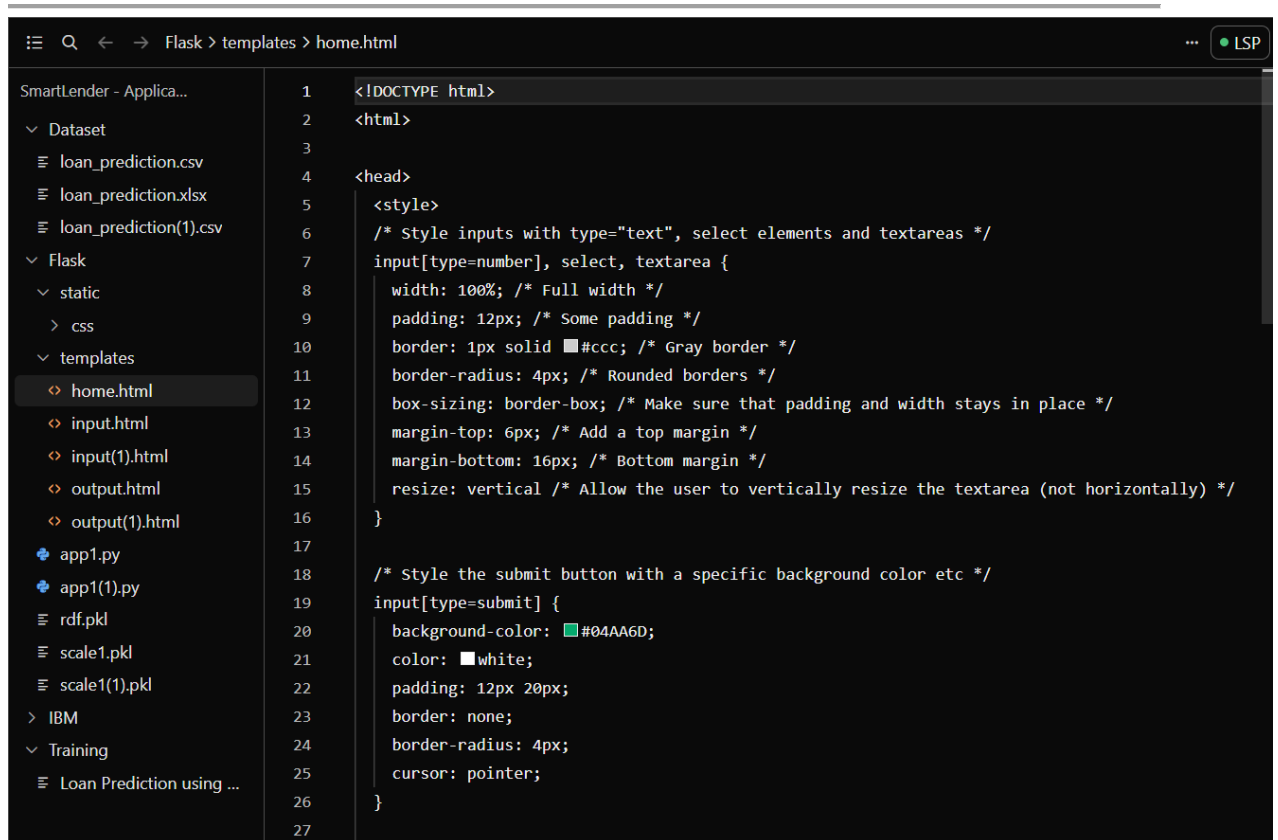
- Create a CSS stylesheet (static/css/style.css) to style the web application.
- Apply a responsive layout using **Flexbox** and **CSS Grid** so that the application works properly on desktop, tablet, and mobile devices.
- Use appropriate colors, fonts, spacing, and buttons to create an intuitive and user-friendly interface.

Create the Loan Application Form

Design a form that allows users to enter the following applicant information:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

Include a **Predict Loan** button to submit the application.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'Dataset', 'Flask', 'static', 'css', 'templates', and 'Training'. The 'templates' folder is expanded, showing files like 'home.html', 'input.html', 'input(1).html', 'output.html', and 'output(1).html'. The 'home.html' file is selected. The code editor shows the following HTML and CSS:

```

1 <!DOCTYPE html>
2 <html>
3
4 <head>
5   <style>
6     /* Style inputs with type="text", select elements and textareas */
7     input[type=number], select, textarea {
8       width: 100%; /* Full width */
9       padding: 12px; /* Some padding */
10      border: 1px solid #ccc; /* Gray border */
11      border-radius: 4px; /* Rounded borders */
12      box-sizing: border-box; /* Make sure that padding and width stays in place */
13      margin-top: 6px; /* Add a top margin */
14      margin-bottom: 16px; /* Bottom margin */
15      resize: vertical /* Allow the user to vertically resize the textarea (not horizontally) */
16    }
17
18    /* Style the submit button with a specific background color etc */
19    input[type=submit] {
20      background-color: #04AA6D;
21      color: white;
22      padding: 12px 20px;
23      border: none;
24      border-radius: 4px;
25      cursor: pointer;
26    }
27  
```

Activity 4.2: Integrating the Flask Backend

Configure Flask Routes

Create Flask routes to navigate between the application pages.

```
@app.route("/")
def home():
    return render_template("home.html")
@app.route("/predict", methods=["GET", "POST"])
def predict():
    # Receive user input
    # Load trained model
    # Predict loan status
    # Display prediction
```

```
<h3>Enter your Details for Loan Approval Prediction</h3>

<div class="container">

    <form action = 'submit', method = 'post'>

        <label for="Gender">Gender</label>
        <select id="Gender" name="Gender">
            <option value=0>Male</option>
            <option value=1>Female</option>
        </select>

        <label for="Married">Married</label>
        <select id="Married" name="Married">
            <option value=1>Yes</option>
            <option value=0>No</option>
        </select>
```

Retrieve User Input

- Collect the applicant information entered in the HTML form using **POST** requests.
- Convert the received values into the format expected by the trained machine learning model.

Load the Trained Model

- Load the saved **XGBoost model (.pkl)** using the **Pickle** library.

```
import pickle

model = pickle.load(open("xgboost_model.pkl", "rb"))
```

Generate Loan Prediction

- Pass the applicant details to the trained model.
- Predict whether the loan application is **Approved** or **Rejected**.
- Display the prediction result on the **submit.html** page.

Validate the Application

- Verify that all input fields are completed correctly.
- Test the application using different applicant details.
- Ensure that prediction results are displayed accurately and without errors.



Milestone 5: Testing and Deployment

In Milestone 5, the Smart Lender application is tested to ensure that all functionalities work correctly before deployment. The machine learning model, Flask backend, and web interface are validated to provide accurate loan eligibility predictions. After successful testing, the application is deployed on IBM Cloud, making it accessible to users through a web browser.

Activity 5.1: Preparing the Application for Local Deployment

Set Up the Development Environment:

- Ensure Python, Flask, and all required machine learning libraries are installed.
- Install all project dependencies listed in the requirements file.
- Verify that the trained machine learning model and dataset are available in the project directory.
- Confirm that the project structure contains the required folders such as **templates**, **static**, **dataset**, and **models**.

Configure the Application:

- Verify that the Flask application is correctly configured.
- Ensure the saved machine learning model is loaded successfully using the Pickle library.
- Check that all required project files are properly connected before running the application.

Activity 5.2: Local Testing and Verification

Run the Smart Lender Application:

- Launch the Flask application in the local development environment.
- Open the application in a web browser using the local server URL.

Test the Application Features:

- Verify that the Home page loads successfully.
- Navigate to the Loan Prediction page.
- Enter different applicant details in the prediction form.
- Submit the form and verify that the application predicts the loan approval status correctly.
- Test multiple combinations of applicant data to validate the accuracy and consistency of the prediction results.

Validate the Application:

- Ensure that all pages are working without errors.
 - Verify that the frontend communicates correctly with the Flask backend.
 - Confirm that the prediction results are displayed accurately on the results page.
 - Check that the application performs smoothly and responds within an acceptable time.
-

Activity 5.3: IBM Cloud Deployment**Prepare the Application for Deployment:**

- Verify that the application is functioning correctly in the local environment.
- Ensure all required project files, dependencies, and the trained machine learning model are included.

Deploy the Application:

- Log in to the IBM Cloud platform.
- Create the deployment environment for the Flask application.
- Upload the Smart Lender project and deploy the application.
- Wait until the deployment process completes successfully.

Verify the Deployment:

- Open the deployed application using the IBM Cloud URL.
 - Test the Smart Lender application by entering applicant details.
 - Confirm that the deployed application generates accurate loan eligibility predictions.
 - Verify that the application is accessible from different devices and web browsers.
-

Important Notes

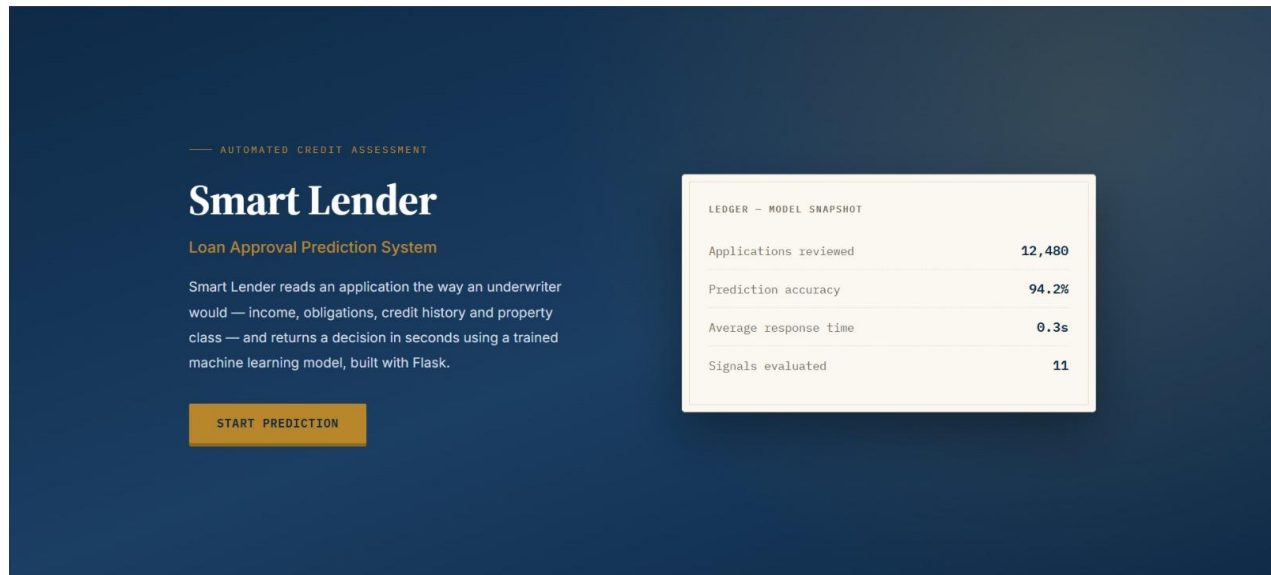
- Ensure that all required Python libraries are installed before deployment.
- Verify that the trained machine learning model is correctly loaded during application startup.
- Test the application thoroughly in the local environment before deploying it to IBM Cloud.
- After deployment, validate that all functionalities work correctly and the prediction results are generated accurately.

Exploring the Website's Web Pages

Home Page

Description:

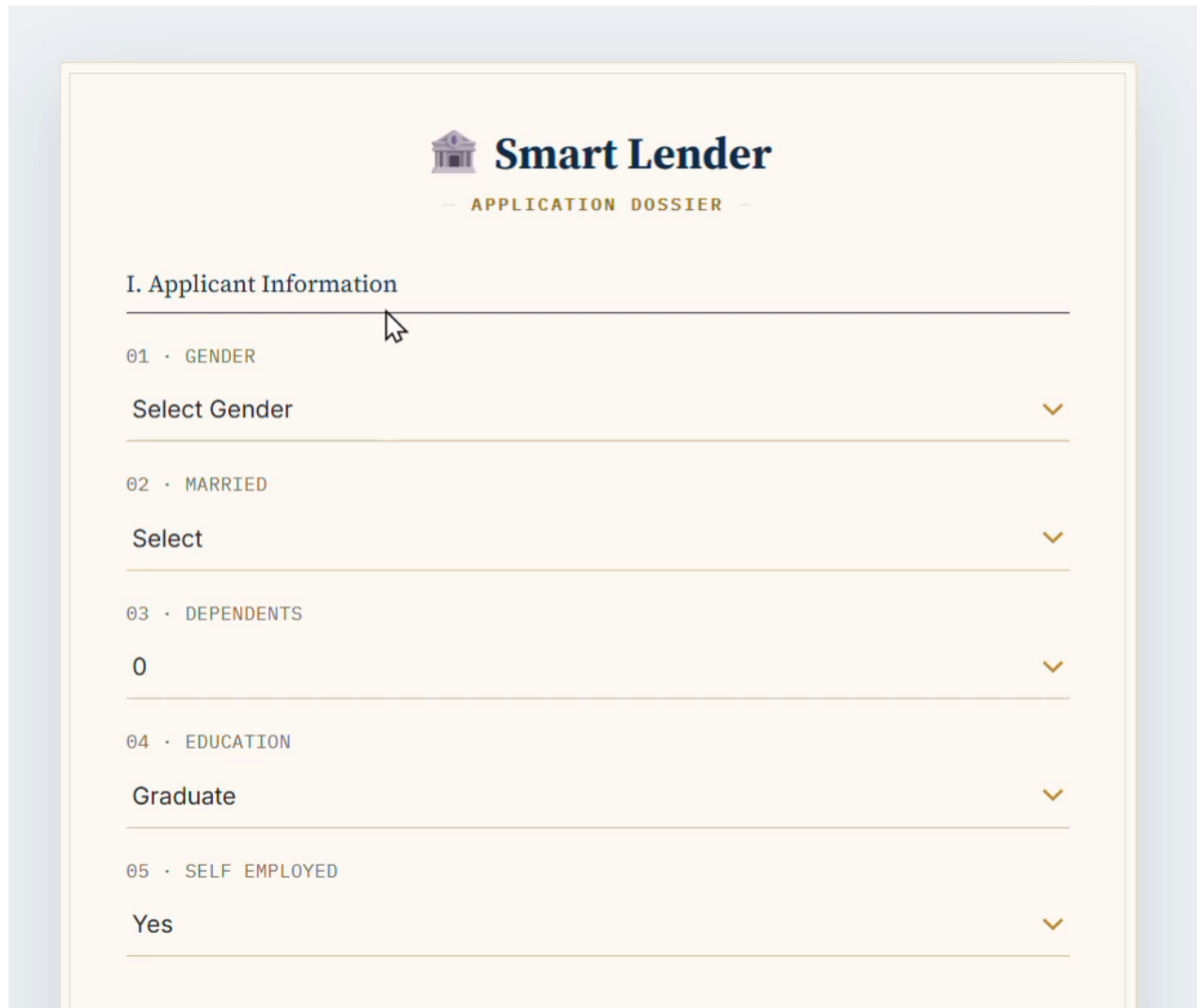
The Home Page serves as the landing page of the Smart Lender application. It provides users with an overview of the system and explains the purpose of the loan approval prediction platform. The page contains a welcoming interface with the project title, a brief description of the application, and a Predict Loan button that directs users to the loan prediction form. The clean and user-friendly layout ensures easy navigation for first-time users.



Loan Prediction Page

Description:

The Loan Prediction page contains the applicant information form where users enter the details required for loan eligibility prediction. The form collects information such as Gender, Marital Status, Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. After entering all the required details, users click the **Predict** button to submit the information to the trained machine learning model.



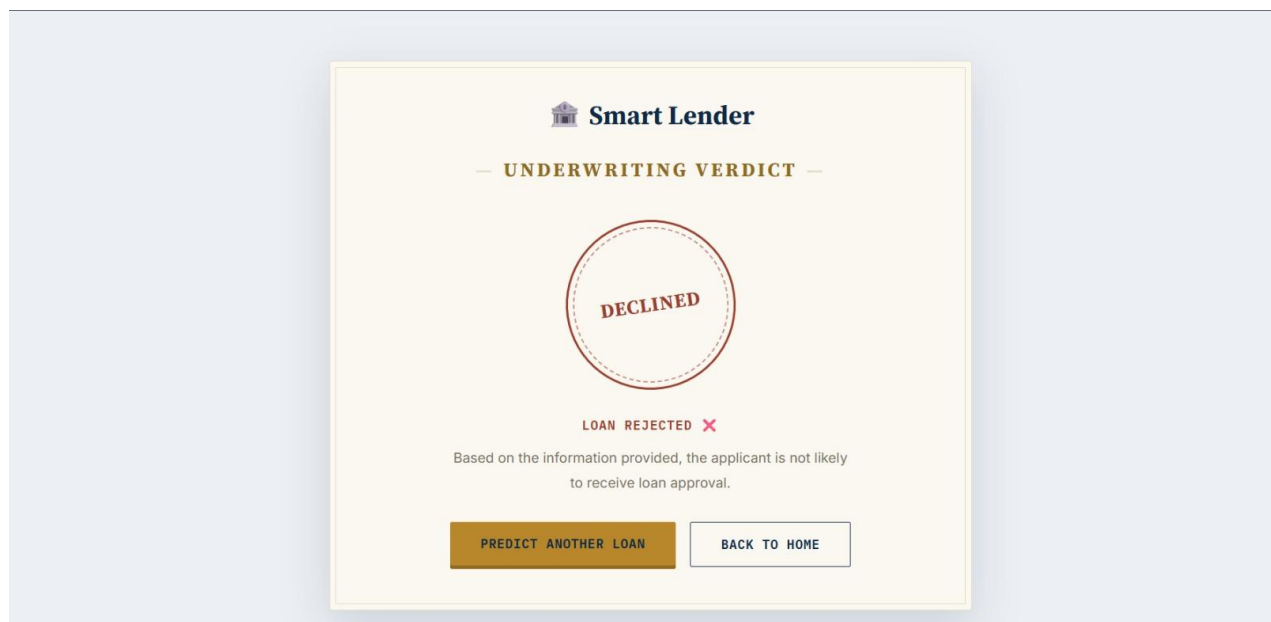
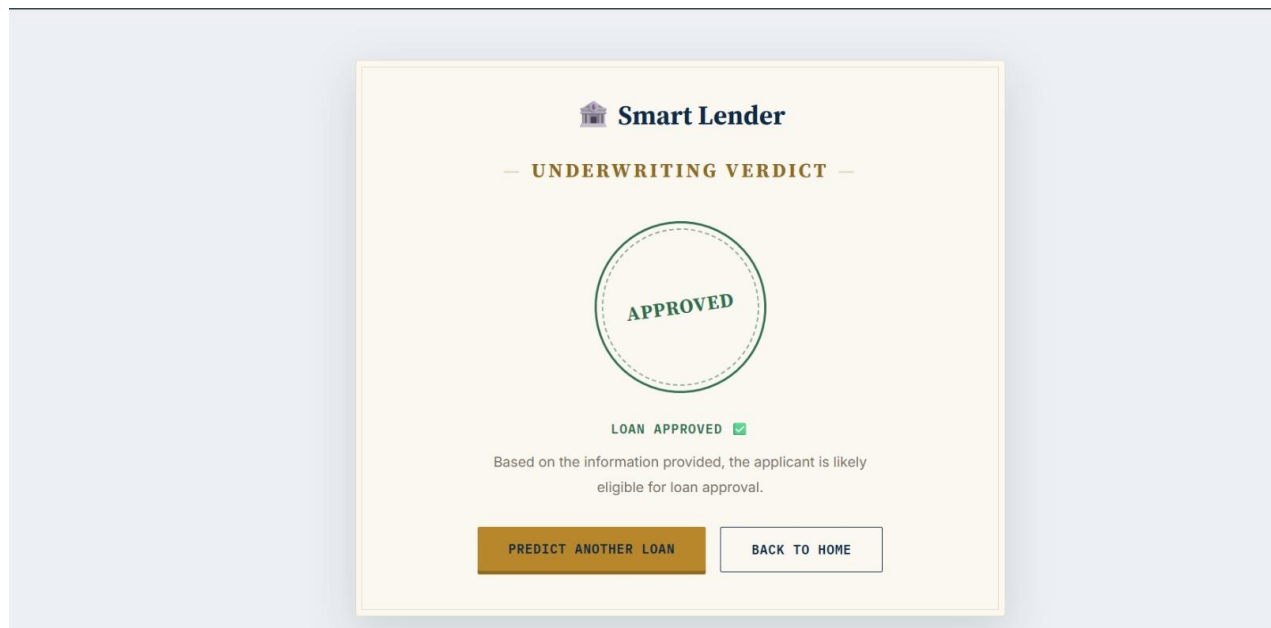
The image shows a screenshot of the 'Smart Lender' application dossier form. At the top, there is a logo for 'Smart Lender' with a house icon and the text 'Smart Lender' in a serif font, followed by 'APPLICATION DOSSIER' in a smaller, all-caps sans-serif font. Below this, the section 'I. Applicant Information' is highlighted. The form contains five sections, each with a dropdown menu:

- 01 • GENDER**: The dropdown menu is open, showing 'Select Gender' with a downward arrow.
- 02 • MARRIED**: The dropdown menu is open, showing 'Select' with a downward arrow.
- 03 • DEPENDENTS**: The dropdown menu is open, showing '0' with a downward arrow.
- 04 • EDUCATION**: The dropdown menu is open, showing 'Graduate' with a downward arrow.
- 05 • SELF EMPLOYED**: The dropdown menu is open, showing 'Yes' with a downward arrow.

Prediction Result Page

Description:

The Prediction Result page displays the output generated by the trained XGBoost machine learning model. Based on the applicant's information, the system predicts whether the loan application is **Approved** or **Rejected**. The result is presented clearly along with an option to return to the prediction page and evaluate another applicant. This page provides users with an instant and accurate loan eligibility prediction.



Conclusion

Smart Lender is a machine learning-based web application developed using Flask to simplify and automate the loan approval process. The system analyzes applicant information such as income, education, employment status, credit history, loan amount, and property area to predict loan eligibility accurately. Multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, were trained and evaluated, with XGBoost selected as the final model due to its superior prediction performance.

The project successfully covered the complete machine learning lifecycle, including data collection, exploratory data analysis, data preprocessing, feature engineering, model training, evaluation, web application development, testing, and deployment. By integrating the trained model with a Flask-based web application, Smart Lender provides users with a simple interface for obtaining instant loan approval predictions, helping financial institutions make faster and more reliable lending decisions.

Looking ahead, the Smart Lender system can be further enhanced by integrating real-time credit score verification, secure user authentication, cloud-based database management, and advanced deep learning models to improve prediction accuracy. Additional features such as loan recommendation systems, customer dashboards, automated report generation, and support for mobile platforms can further increase the system's usability and effectiveness. These enhancements will enable Smart Lender to evolve into a comprehensive intelligent loan assessment platform for modern banking and financial institutions.